



Long-Context Compression Primitives for Extension Protocol Architects

1. Conceptual Map

Across the literature you specified, long-context efficiency falls into three distinct layers:

1. Lexical / prompt-level compression

- LLMingua, LongLLMingua, LLMingua-2: drop redundant tokens while preserving semantics. [\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#) [\[5\]](#) [\[6\]](#) [\[7\]](#) [\[8\]](#) [\[9\]](#) [\[10\]](#)
- Chain of Density (CoD): iterative abstractive summarization that increases entity density at fixed length. [\[11\]](#) [\[12\]](#) [\[13\]](#) [\[14\]](#) [\[15\]](#) [\[16\]](#) [\[17\]](#)

2. Representation-level compression (inside / near the model)

- Activation Beacon: compresses raw activations (KV cache) into a small set of beacon tokens; extends context up to ~100x. [\[18\]](#) [\[19\]](#) [\[20\]](#) [\[21\]](#) [\[22\]](#) [\[23\]](#) [\[24\]](#) [\[25\]](#) [\[26\]](#)
- xRAG: treats document embeddings as a separate “retrieval modality” and fuses them into the LM as 1-token context surrogates. [\[27\]](#) [\[28\]](#) [\[29\]](#)

3. Retrieval / indexing-level compression

- Semantic vs fixed-size chunking (Vectara + follow-ups): large-scale evaluation shows fixed-size chunking often matches or beats semantic chunking in realistic RAG, with far lower cost. [\[30\]](#) [\[31\]](#) [\[32\]](#) [\[33\]](#) [\[34\]](#) [\[35\]](#) [\[36\]](#)

For extension protocols, the key is to expose these as **orthogonal knobs**—decoupling:

- *What to keep* (salient entities / tokens / chunks),
- *Where to encode it* (text vs embeddings vs activations),
- *When to apply compression* (ingestion, retrieval, pre-prompt, or during generation).

The sections below walk through each primitive, then close with integration patterns you can wire into any of your “cats”.

2. LLMingua Series – Prompt-Side Compression

2.1 LLMingua (EMNLP 2023)

LLMingua’s core idea: **a small LM learns which tokens in a prompt are dispensable**, so the large LM can operate on a compressed prompt with minimal loss. [\[2\]](#) [\[4\]](#) [\[8\]](#) [\[1\]](#)

Mechanism (original LLMingua): [\[4\]](#) [\[8\]](#) [\[1\]](#) [\[2\]](#)

- Use a smaller LM (e.g., GPT-2 small or LLaMA-7B) to estimate token importance.
- Two-stage compression:

1. **Coarse-grained (sentence-level)**: drop low-importance sentences or segments.
 2. **Fine-grained (token-level)**: iteratively remove unimportant tokens while preserving coherence (budget controller enforces a target compression rate).
- Closed LLM (e.g., GPT-3.5) never retrained; it simply consumes the compressed prompt.

Empirical highlights: [\[3\]](#) [\[8\]](#) [\[1\]](#) [\[2\]](#) [\[4\]](#)

- Up to **20x compression** on reasoning, summarization, and dialog datasets (GSM8K, BBH, ShareGPT, Arxiv-March23) with **little performance loss**.
- Non-intuitive but validated: compressed prompts look “garbled” to humans yet remain interpretable for LLMs.
- Also reduces **generated output length**, compounding cost savings.

2.2 LongLLMLingua – for Long Context / “Lost in the Middle”

LongLLMLingua adapts LLMingua to **long-context QA / RAG**, targeting three issues: high cost, performance degradation with long prompts, and position bias. [\[9\]](#) [\[10\]](#)

Key ideas: [\[10\]](#) [\[9\]](#)

- Optimize **density and position** of key information in long prompts; mitigate “lost in the middle” by reordering/squeezing salient spans toward positions the LM attends to more reliably.
- Introduces a **post-compression subsequence recovery strategy** to avoid deleting critical connective text after aggressive compression. [\[10\]](#)
- Evaluated across **NaturalQuestions, LongBench, ZeroSCROLLS, MuSiQue, LooGLE**: [\[9\]](#) [\[10\]](#)
 - Compressed prompts **often outperform** original prompts while using fewer tokens.
 - Significant **cost and latency reductions** in long-context regimes.

2.3 LLMLingua-2 – Distilled, Task-Agnostic Compressor

LLMLingua-2 reframes prompt compression as **token classification**, trained via data distillation from GPT-4. [\[37\]](#) [\[5\]](#) [\[6\]](#) [\[7\]](#) [\[2\]](#)

Mechanism: [\[5\]](#) [\[6\]](#) [\[7\]](#) [\[37\]](#)

- Construct an **extractive text compression dataset** by having GPT-4 perform high-quality extractive compression.
- Train a **BERT/XLM-RoBERTa-class encoder** to predict for each token: *keep vs drop*.
- Uses full **bidirectional context**, unlike entropy-based baselines that only see left context. [\[7\]](#) [\[37\]](#) [\[5\]](#)

Results: [\[38\]](#) [\[6\]](#) [\[2\]](#) [\[3\]](#) [\[37\]](#) [\[5\]](#) [\[7\]](#)

- **2–5x compression**, maintaining task performance across in-/out-of-domain datasets: MeetingBank, LongBench, ZeroScrolls, GSM8K, BBH. [\[6\]](#) [\[37\]](#) [\[5\]](#) [\[7\]](#)
- **3–6x faster** than prior prompt-compression methods, with **1.6–2.9x E2E latency reduction** at similar quality. [\[37\]](#) [\[5\]](#) [\[6\]](#)
- More robust to **out-of-domain prompts** than original LLMLingua. [\[2\]](#) [\[3\]](#)

2.4 What This Gives an Extension Protocol

As extension infrastructure, you can treat the LLMingua series as **prompt filters** with tunable budgets:

- **Capabilities to expose**
 - `compress_prompt(prompt, rate)` – LLMingua-2/LongLLMingua with target compression rate.
 - `analyze_saliency(prompt)` – get token-/span-level importance scores to drive routing (e.g., “only run heavy tools on high-saliency sections”).
- **Routing policies**
 - For **short prompts** (<1–2k tokens): skip compression except on cost-sensitive paths.
 - For **long or RAG-style prompts**:
 - Run LongLLMingua with a modest budget (e.g., 2–4× compression) before sending to the main LM.
 - Optionally re-run on *retrieved* context blocks before concatenation.
- **Key invariants**
 - Avoid extreme compression (>10×) for **creative generation** or tasks that rely on exact wording; stay in 2–5× range for safety. [\[1\]](#) [\[3\]](#) [\[5\]](#) [\[6\]](#) [\[2\]](#) [\[37\]](#)
 - For long-context QA, pair LongLLMingua with **position-aware layouts** (e.g., keep highest-salience spans nearer the end of the prompt to combat position bias). [\[9\]](#) [\[10\]](#)

3. Chain of Density (CoD) – Entity-Dense Summarization

CoD is not “compression” in the LLMingua sense, but a **prompting protocol that converts a source into a series of increasingly entity-dense summaries at fixed length**. [\[12\]](#) [\[13\]](#) [\[14\]](#) [\[15\]](#) [\[16\]](#) [\[17\]](#) [\[11\]](#)

3.1 Mechanism

The CoD prompt asks GPT-4 (or equivalent) to: [\[13\]](#) [\[39\]](#) [\[40\]](#) [\[16\]](#) [\[17\]](#)

1. Generate an initial, **entity-sparse** summary (~4–5 sentences, ~70–80 words) with only **1–3 salient entities**.
2. For each of 4 further steps (total 5 summaries):
 - Identify 1–3 **Missing Entities** (relevant, specific ≤5 words, novel, faithful, anywhere in article). [\[39\]](#) [\[14\]](#) [\[40\]](#) [\[13\]](#)
 - Write a new **Denser_Summary** of **identical length**, incorporating:
 - All previous entities, plus
 - The new Missing Entities,
 - While using abstraction, fusion, and compression to make space. [\[14\]](#) [\[40\]](#) [\[13\]](#)
3. Output JSON: list of 5 items, each with “Missing_Entities” and “Denser_Summary”. [\[40\]](#) [\[13\]](#) [\[39\]](#)

Internally, density is proxied by **entity-per-token ratio**.

3.2 Empirical Findings

From the CoD paper (CNN/DailyMail, 100 articles):^{[16] [17] [13]}

- **Entity density:**
 - Vanilla GPT-4 summaries: density ≈ 0.122 entities/token.
 - Human summaries: ≈ 0.151 .
 - CoD summaries: increase from 0.089 at step 1 to 0.167 at step 5.^{[17] [13]}
- **Human preferences:**
 - Human annotators preferred CoD summaries **more dense than vanilla GPT-4** and **almost as dense as human-written**.^{[13] [17]}
 - Median preferred step ≈ 3 (out of 5) – too sparse (step 1) and too dense (step 5) both underperform.^{[41] [11] [13]}
- **Qualitative behavior:**
 - CoD summaries are **more abstractive**, show more **fusion**, and have **less lead bias** (they pull entities from the middle and end, not just the opening sentences).^{[17] [13]}

This makes CoD an excellent *semantic compression* protocol for long documents where you want **fixed-length, high-density, human-friendly** representations.

3.3 Integration Patterns

For your stack, CoD is a **summarization primitive** you can standardize:

- **Summarization agent:**
 - `summarize_with_CoD(text, steps=n)` returns the list of summaries and entity-sets.
 - Default to **step 2–3** as the “production summary” based on human preference studies.^{[11] [13]}
- **As a compression stage**
 - For very long context, instead of concatenating all retrieved chunks, **CoD-summarize** each document into a fixed-length summary (80–120 words).
 - Then apply **LLMLingua-2** to these summaries if you still need to reduce tokens further; you get a “double compression” stack:
 - CoD → semantics-dense text,
 - LLMLingua-2 → token-efficient representation.
- **For memory/state**
 - Use CoD step 3 summaries as **canonical memory traces** for episodes/conversations, giving your agents a high-density but human-readable memory substrate.

4. xRAG – Extreme Context Compression in RAG

xRAG (NeurIPS 2024) is a **representation-level compression** method that treats document embeddings as a separate input modality and fuses them into the LM as single tokens.^{[28] [29] [27]}

4.1 Core Mechanism

Traditional RAG: [\[29\]](#) [\[27\]](#) [\[28\]](#)

- Query → retriever → top-k documents → concatenate full or chunked text into prompt.
- Cost scales with **sum of retrieved text length**.

xRAG instead: [\[27\]](#) [\[28\]](#) [\[29\]](#)

1. Use a dense retriever as usual to get **document embeddings**.
2. Treat each doc embedding as a **retrieval-modality feature** (like an image embedding in CLIP-style multimodal LMs).
3. Learn a **modality bridge** (the only trainable component, <0.1% of LM params) that maps retrieval embeddings into the LM's representation space. [\[28\]](#)
4. At generation time, feed:
 - The **query tokens**,
 - **One token per document** representing the fused embedding, instead of the full text. [\[29\]](#) [\[27\]](#) [\[28\]](#)

So instead of adding thousands of tokens of retrieved context, you add **O(k)** tokens, where k ≈ number of docs.

4.2 Results and Trade-offs

Findings: [\[27\]](#) [\[28\]](#) [\[29\]](#)

- **>10% average improvement** over strong baselines across six knowledge-intensive tasks.
- **Comparable to uncompressed RAG** on several benchmarks, while:
 - Reducing total FLOPs by **≈3.5×**.
 - Achieving an **order-of-magnitude effective compression** (doc → 1 token + query).
- Retriever and LM are **frozen**; only the modality bridge is trained, preserving plug-and-play retrieval.

Implications:

- xRAG's compression is **non-extractive**: the LM must learn to decode dense embeddings back into useful information.
- This is a **model architecture / fine-tuning** primitive; you cannot bolt it on purely at the prompt level.

4.3 How to Exploit xRAG Conceptually

Unless you're training your own LM variants, xRAG is more of a **design pattern** than a direct tool:

- For extension protocols:
 - Treat retrieved documents as **embeddings first**, text second.
 - Architect the system so downstream components can operate on:
 - doc_text (for standard RAG),
 - or doc_embedding (for xRAG-style or other embedding-level fusions).
 - When/if xRAG-like models become available as APIs, your routing can do:
 - `if model_supports_embedding_fusion: send embeddings only; else: send compressed text (LLMLingua-2 + CoD).`

- Even without xRAG, **embedding-only reasoning** (e.g., using an auxiliary small LM that reads embeddings or cluster centroids) can approximate some benefits.

5. Activation Beacon – Long Context Compression in the KV Space

Activation Beacon is a **plug-in module** for transformer LLMs that compresses long contexts *inside the model*, at the level of keys/values in attention. [\[19\]](#) [\[20\]](#) [\[21\]](#) [\[22\]](#) [\[23\]](#) [\[42\]](#) [\[24\]](#) [\[25\]](#) [\[26\]](#) [\[43\]](#) [\[18\]](#)

5.1 Mechanism

Key pieces: [\[20\]](#) [\[21\]](#) [\[22\]](#) [\[23\]](#) [\[24\]](#) [\[25\]](#) [\[26\]](#) [\[18\]](#) [\[19\]](#)

- Introduce a special **beacon token** (b).
- Partition the long input into **chunks** (e.g., 1024–2048 tokens).
- Within each chunk:
 - Further split into fine-grained units of size α (related to desired compression ratio).
 - Interleave beacons at the end of each unit.
 - During self-attention, **distill** the unit's activations (keys/values) into beacon activations.
- After encoding each chunk:
 - **Discard** raw token activations from that chunk.
 - **Accumulate** beacon activations.
- When encoding the next chunk:
 - Tokens attend to **current raw tokens + accumulated beacon activations** as a compact representation of all prior chunks. [\[21\]](#) [\[20\]](#)

Training: [\[25\]](#) [\[20\]](#) [\[21\]](#)

- Beacons are learned **post-hoc** on top of a frozen LM via standard autoregressive training.
- Trained with sliding windows and mixed compression ratios so the model learns to rely on beacon-compressed context.
- Beacons mostly reuse the original LM parameters; the additional module is **parameter-efficient**.

5.2 Results

Across long-context tasks (document understanding, few-shot with long context, Needle-in-a-Haystack), Activation Beacon: [\[26\]](#) [\[18\]](#) [\[19\]](#) [\[20\]](#) [\[21\]](#) [\[25\]](#)

- Extends LLaMA-2-7B's context from **4K to ~400K** tokens ($\approx 100\times$). [\[21\]](#) [\[25\]](#)
- Achieves **comparable performance** to full, uncompressed models on many long-context benchmarks. [\[18\]](#) [\[20\]](#) [\[21\]](#)
- Delivers up to **2× inference speedup** and **8× reduction in KV-cache memory** at similar quality. [\[20\]](#) [\[18\]](#)
- Maintains **short-context capabilities** close to the original LM; the plug-and-play design avoids degrading base abilities. [\[25\]](#) [\[20\]](#) [\[21\]](#)

5.3 Relevance for Extension Protocols

Practically, this is a **model-level capability**, not a prompt trick. But for architecture planning:

- Activation Beacon is conceptually a **state compressor for KV memory**. For your agents:
 - Treat beacon-compressed models as having a **massively larger but coarser “episodic buffer”**.
 - You can design routing like:
 - Use standard models + RAG for short/medium context.
 - Route very long, document-heavy reasoning (400k+) to Beacon-augmented models when available.
- Analogy for your own infra:
 - When you can’t modify the base LM, approximate “activation beacons” via:
 - **Hierarchical summaries + persistent embeddings** that survive across turns,
 - Minimal raw text repetition in prompts, relying more on these compressed state representations.

6. Semantic Chunking vs Fixed-Size Chunking (Vectara Study)

Vectara’s 2024–2025 study is the strongest empirical comparison of **semantic vs fixed-size chunking** in RAG. [\[31\]](#) [\[32\]](#) [\[34\]](#) [\[35\]](#) [\[36\]](#) [\[30\]](#)

6.1 Experimental Setup

They define three chunkers: [\[32\]](#) [\[34\]](#) [\[30\]](#)

1. Fixed-size Chunker:

- Sequentially splits doc into chunks with a fixed **number of sentences**, optional 0/1 sentence overlap.

2. Breakpoint-based Semantic Chunker:

- Computes semantic distance between adjacent sentence embeddings.
- Inserts a breakpoint when distance exceeds a threshold (topic change).

3. Clustering-based Semantic Chunker:

- Clusters sentences based on a hybrid distance (positional + semantic) and groups them into chunks. [\[34\]](#) [\[32\]](#)

Tasks: [\[30\]](#) [\[32\]](#) [\[34\]](#)

- **Document retrieval**: map retrieved chunks back to docs, evaluate doc-level retrieval.
- **Evidence retrieval**: locate gold evidence sentences, evaluate F1@5.
- **Answer generation**: GPT-4o generates answers based on retrieved chunks, and output quality is scored.

Datasets: [\[32\]](#) [\[34\]](#) [\[30\]](#)

- Constructed **stitched documents** (from BEIR) to artificially increase topic diversity.
- Used multiple realistic datasets (RAGBench and others) where documents are not stitched.

6.2 Key Findings

Core conclusions: [\[35\]](#) [\[36\]](#) [\[34\]](#) [\[30\]](#) [\[32\]](#)

- **Minimal performance differences** overall between fixed-size and semantic chunking.
- On **stitched, highly diverse documents**, breakpoint-based semantic chunking sometimes wins (better topic integrity). [\[32\]](#)
- On **realistic, non-stitched data, fixed-size chunking often outperforms semantic chunking**, especially for document retrieval and evidence retrieval (F1@5). [\[34\]](#) [\[30\]](#) [\[32\]](#)
- For GPT-4o answer generation, differences were **negligible**—LLMs are good at fishing relevant content out of “noisy” chunks. [\[30\]](#) [\[34\]](#) [\[32\]](#)
- **Computational cost** of semantic chunking is substantial:
 - embedding every sentence,
 - running thresholding or clustering,
 - often not justified by small and inconsistent performance gains. [\[44\]](#) [\[30\]](#) [\[32\]](#)

Meta-implications: [\[36\]](#) [\[31\]](#) [\[35\]](#) [\[34\]](#) [\[30\]](#) [\[32\]](#)

- **Embedding quality** dominates: with strong embeddings, chunking strategy matters less.
- Fixed-size chunking with good hyperparameters (chunk sizes, overlaps) offers:
 - **Simplicity and scalability**,
 - **Consistency** across datasets,
 - Adequate retrieval for downstream LLMs to fix via generation.

Other work (e.g., SemRAG) finds semantic chunking can help in certain hierarchical graph-RAG setups, but even there **fixed-size chunks often win on correctness while semantic chunks may help relevancy**. [\[33\]](#)

6.3 Guidance for Your RAG Pipelines

For an extension framework:

- Default to **fixed-size chunking**:
 - E.g., 150–300 tokens with 10–20% overlap for text; adjust per domain.
- Reserve **semantic chunking** for:
 - Highly heterogeneous documents where you know topic shifts are frequent (e.g., stitched corpora, aggregated reports).
 - Specialized graph-RAG or SemRAG-style pipelines where the extra structure is explicitly exploited. [\[33\]](#)
- Make chunking **pluggable**:
 - Expose both “fixed” and “semantic” strategies behind a `chunker` interface and treat semantic chunkers as opt-in, not default.

7. Comparative Snapshot

7.1 Techniques Across the Stack

Method	Layer / Scope	Mechanism (very short)	Typical Compression / Effect	Training Need	Pros	Cons / Caveats
LLMLingua	Prompt text	Small LM removes low-importance tokens/sentences via budgeted compression [1] [2] [4] [8]	Up to ~20× tokens vs original prompt [1] [2] [3] [4]	Train small LM; big LM frozen	Large savings; good for RAG, reasoning; no big-LM retrain	Very aggressive ratios risky; original LLMLingua less robust OOD than v2
LongLLMLingua	Long-context prompts / RAG	Position- and density-aware compression; subsequence recovery [9] [10]	2–4×+ with better performance [9] [10]	Extra training on long-context data	Mitigates lost-in-the-middle; boosts long QA	More complex, tailored to QA-like tasks
LLMLingua-2	General prompt compression	BERT/XLM-R token classifier distilled from GPT-4 [37] [5] [6] [7]	2–5×, 3–6× faster than prior methods [37] [5] [6]	Train small encoder; big LM frozen	Task-agnostic, robust OOD, plug-and-play	Still needs training infra; extreme compression still non-trivial
Chain of Density	Summarization (semantic)	Iterative fixed-length entity-dense summaries [13] [16] [17]	Same length, ↑ entity density to ~human level [13] [17]	No training; prompt only	Human-preferred density; less lead bias; more abstractive	Multiple calls per doc; tuned to summarization, not arbitrary prompts
xRAG	RAG context (representation-level)	Modality fusion of doc embeddings as 1-token features [28] [27] [29]	Docs → 1 token each; ≈3.5× FLOPs reduction [28]	Train small modality bridge	Huge context savings; matches uncompressed RAG on many tasks	Requires specialized LM; black-box APIs rarely expose it (yet)
Activation Beacon	LM internal KV cache	Beacon tokens condense activations across chunks [20] [21] [25]	≈100× context (4K → 400K); 2× speed; 8× KV saving [18] [20] [21] [25]	Train beacon module post-hoc	Massive context, low extra params, preserves base LM	Requires LM weights; not a generic API trick
Fixed-size chunking	Retrieval index	N-sentence or N-token windows w/ small overlap [30] [31] [32] [35]	No “compression”; defines retrieval grain	None	Simple, scalable, often best in practice	Can fragment topics; but LLMs often compensate
Semantic chunking	Retrieval index	Breakpoints or clustering on sentence embeddings [30] [31] [32] [33]	Similar avg chunk size, higher compute	None (rule-based)	Can help on synthetic/highly diverse docs	Inconsistent gains, expensive; often worse on real corpora [30] [32] [34]

8. Design Patterns for Your Extension Protocols

Finally, how to wire these into a **generic, model-agnostic extension layer** that any of your agents/cats can invoke.

8.1 Multi-Layer Compression Policy

Think in three passes:

1. Ingestion / Indexing (retrieval layer)

- Use **fixed-size chunking** as default, with tunable sizes; allow `semantic_chunking` only when explicitly requested by a task profile. [\[35\]](#) [\[34\]](#) [\[30\]](#) [\[32\]](#)
- Store both:
 - **Raw text chunks**, and
 - **High-quality embeddings** (e.g., stella, BGE, etc.); the embeddings are the substrate for:
 - Standard similarity search,
 - Future xRAG-style embedding fusion.

2. Pre-RAG Compression (document side)

- Optionally:
 - Run **CoD summarization** per document to obtain canonical dense summaries.
 - Store summaries alongside original chunks as alternative retrieval payloads.
- For extremely large corpora, have a “fast path” where **only CoD summaries** are indexed for some tasks.

3. Prompt Construction (query side)

- Before hitting the main LM:
 - If total token count > threshold, route through **LLMLingua-2 / LongLLMLingua** with a moderate budget (e.g., 2–4×).
 - For long QA / multi-hop reasoning over retrieved context, always enable **LongLLMLingua** with position-aware compression. [\[10\]](#) [\[9\]](#)

8.2 Routing Heuristics for Cats/Agents

You can encode policies like:

- **Cat: “Summarizer”**
 - Always use **CoD**:
 - Step 3 summary as output default (with option to expose all 5 steps as a density slider to the user). [\[11\]](#) [\[13\]](#)
 - If source > X tokens, CoD + LLMLingua-2 on the summaries to meet hard token caps.
- **Cat: “Research QA / RAG”**
 - Retrieval:
 - Fixed-size chunking only, with tuned chunk size per domain. [\[34\]](#) [\[30\]](#) [\[32\]](#)
 - Context assembly:
 - Use raw chunk text when answers require verbatim citations.

- Use **CoD summaries of top-k docs** for “analysis” style answers.
- Pre-prompt compression:
 - LongLLMLingua pre-processing when query + context > threshold, especially on long benchmarks (NaturalQuestions, LongBench-like tasks). [\[9\]](#) [\[10\]](#)
- **Cat: “Long Context Reasoner / Workspace Agent”**
 - Prefer models with **Activation Beacon** or equivalent long-context extensions when available.
 - Maintain a **two-tier memory**:
 - Tier 1: recent raw turns (no compression).
 - Tier 2: CoD summaries of past sections + embeddings (for retrieval).
 - Only send Tier 2 into the context after **LLMLingua-2 compression**, effectively mimicking “beacon-style” state compression at the textual level.
- **Cat: “Cost-Constrained Inference”**
 - For any task:
 - If cost budget low and text long:
 - Force **LLMLingua-2 with aggressive rate** (up to 5×), with a warning if task is highly sensitive to exact wording (e.g., legal, code). [\[5\]](#) [\[6\]](#) [\[37\]](#)
 - Optionally replace retrieved full texts with **CoD summaries** only.

8.3 Extensibility Hooks

To future-proof:

- **Abstract APIs** rather than hard-wiring specific tools:
 - `compress_prompt(text, budget, mode=[lexical|semantic|representation])`
 - `summarize(text, density_target)`
 - `embed(doc) -> embedding`
 - `fuse_embeddings_into_lm(embeddings)` – currently a no-op unless you have xRAG-style models, but defines a slot.
- **Metadata propagation**
 - Tag each compressed artifact with:
 - Original length, compression ratio, method used, saliency map.
 - Let downstream agents decide when to “decompress” (e.g., fetch original text from index when fidelity is critical).

9. Takeaways for Extension-Protocol Experts

1. Prompt compression is real, robust, and production-ready.

LLMLingua-2 + LongLLMLingua give you 2–5× savings with little or negative quality loss across many tasks—these should be first-class citizens in any token-economical protocol. [\[8\]](#) [\[3\]](#) [\[6\]](#) [\[7\]](#) [\[1\]](#) [\[2\]](#) [\[37\]](#) [\[5\]](#) [\[10\]](#) [\[9\]](#)

2. CoD is the gold standard for controllable summary density.

It is cheap to adopt (prompt-only), well-understood empirically, and ideal as a canonical summarization primitive and a high-density “semantic buffer” for long workflows. [\[14\]](#) [\[16\]](#) [\[13\]](#) [\[17\]](#) [\[11\]](#)

3. **Representation-level compressions (xRAG, Activation Beacon) are architectural, not prompt tricks.**

They show what is possible when you can modify or fine-tune the LM: context compression of 10–100× with comparable performance. Design your protocols so they can **attach to these capabilities when models exposing them become accessible**. [18] [20] [21] [25] [28] [29] [27]

4. **Chunking is less glamorous but more important than hype suggests.**

Vectara's study shows that simple fixed-size chunking is extremely competitive and often superior in realistic settings, and semantic chunking's extra cost is rarely justified. Optimize your embeddings and retrieval strategy before over-investing in semantic chunking logic. [30] [32] [34]

5. **The right abstraction is: compression as a multi-layered, policy-driven service.**

View LLMingua, CoD, xRAG, Activation Beacon, and chunking strategies as **orthogonal, composable operators** that can be conditionally applied based on:

- context length,
- task type,
- cost/latency budget,
- model capabilities.

If you want, a next step could be to design **concrete CEP-style operator specs** (inputs/outputs + contracts) for each of these primitives, ready to drop into your orchestrator YAML or n8n nodes.



1. <https://www.microsoft.com/en-us/research/blog/llmlingua-innovating-llm-efficiency-with-prompt-compression/>
2. <https://github.com/microsoft/LLMLingua>
3. <https://www.microsoft.com/en-us/research/project/llmlingua/>
4. <https://www.microsoft.com/en-us/research/publication/llmlingua-compressing-prompts-for-accelerated-inference-of-large-language-models/>
5. <https://arxiv.org/abs/2403.12968>
6. <https://www.microsoft.com/en-us/research/publication/llmlingua-2-data-distillation-for-efficient-and-faithful-task-agnostic-prompt-compression/>
7. <https://llmlingua.com/llmlingua2.html>
8. <https://llmlingua.com/llmlingua.html>
9. <https://arxiv.org/abs/2310.06839>
10. <https://www.microsoft.com/en-us/research/project/llmlingua/longllmlingua/>
11. <https://www.prompthub.us/blog/better-summarization-with-chain-of-density-prompting>
12. <https://www.kdnuggets.com/unlocking-gpt-4-summarization-with-chain-of-density-prompting>
13. <https://pmc.ncbi.nlm.nih.gov/articles/PMC11419567/>
14. <https://www.ssw.com.au/rules/chain-of-density/>
15. <https://github.com/richawo/chain-of-density>
16. <https://aclanthology.org/2023.newsum-1.7/>
17. <https://arxiv.org/abs/2309.04269>
18. <https://arxiv.org/abs/2401.03462>
19. http://playbigdata.ruc.edu.cn/dou/publication/2025_ICLR_Activation_Beacon.pdf
20. <https://arxiv.org/pdf/2401.03462.pdf>
21. <https://arxiv.org/html/2401.03462v1>

22. https://www.linkedin.com/posts/sidreddy_the-paper-soaring-from-4k-to-400k-extending-activity-7153748147364581376-3W_U
23. <https://aimodels.substack.com/p/long-context-compression-with-activation>
24. <https://www.youtube.com/watch?v=0c51bFXuEB8>
25. <https://huggingface.co/papers/2401.03462>
26. https://www.reddit.com/r/LocalLLaMA/comments/1927ge4/soaring_from_4k_to_400k_extending_llms_context/
27. <https://www.themoonlight.io/tw/review/xrag-extreme-context-compression-for-retrieval-augmented-generation-with-one-token>
28. https://proceedings.neurips.cc/paper_files/paper/2024/file/c5cf13bfd3762821ef7607e63ee90075-Paper-Conference.pdf
29. <https://www.semanticscholar.org/paper/xRAG:-Extreme-Context-Compression-for-Generation-Cheng-Wang/38fcc3667a907d6c94267c674aad114aae68441e>
30. <https://www.vectara.com/blog/is-semantic-chunking-worth-the-computational-cost>
31. <https://www.vectara.com/blog/grounded-generation-done-right-chunking>
32. <https://arxiv.org/html/2410.13070v1>
33. <https://arxiv.org/html/2507.21110v1>
34. <https://aclanthology.org/2025.findings-naacl.114/>
35. <https://www.vectara.com/blog/retrieval-augmented-generation-rag-done-right-retrieval>
36. <https://www.vectara.com/blog/towards-a-gold-standard-for-rag-evaluation>
37. <https://aclanthology.org/2024.findings-acl.57/>
38. <https://bdtechtalks.substack.com/p/the-fastest-and-most-efficient-prompt>
39. <https://gist.github.com/thistleknot/73ec788f6f5c1c1f11356c052df07ae0>
40. <https://gist.github.com/harperreed/b7b1ae56df4a269925e0668b823693eb>
41. <https://montrealetics.ai/from-sparse-to-dense-gpt-4-summarization-with-chain-of-density-prompting/>
42. <https://github.com/ggerganov/llama.cpp/issues/5800>
43. <https://www.semanticscholar.org/paper/Long-Context-Compression-with-Activation-Beacon-Zhang-Liu/f69f494ab691481ec353da4be480b334fada6607>
44. https://www.linkedin.com/posts/awadallah_is-semantic-chunking-worth-the-computational-activity-7269433689321725953-6y6T
45. <https://www.ewsolutions.com/semantic-chunking/>
46. <https://www.vectara.com/blog/category/research>
47. https://www.youtube.com/watch?v=HE22y_UVvxY
48. <https://www.newline.co/@zaoyang/fixed-size-chunking-in-rag-pipelines-a-guide--af509f11>
49. <https://arxiv.org/abs/2410.13070>
50. <https://www.qeios.com/read/M7YKDZ>
51. <https://community.databricks.com/t5/technical-blog/the-ultimate-guide-to-chunking-strategies-for-rag-applications/ba-p/113089>
52. <https://openreview.net/forum?id=1eQT9OzfNQ>
53. <https://liner.com/review/long-context-compression-with-activation-beacon>
54. <https://www.youtube.com/watch?v=oKcwaUtp59I>
55. <https://aclanthology.org/2025.acl-long.1500.pdf>
56. <https://microsoft.github.io/promptflow/integrations/tools/llmlingua-prompt-compression-tool.html>
57. <https://github.com/atjsh/llmlingua-2.js>
58. <https://github.com/AIAnytime/Prompt-Compression-using-LLMLingua>

59. <https://github.com/voiceflow-gallagan/llmlingua-api>
60. <https://www.llmlingua.com>
61. <https://docs.langchain.com/oss/python/integrations/retrievers/llmlingua>
62. https://learnprompting.org/docs/advanced/self_criticism/chain-of-density
63. https://www.reddit.com/r/PromptEngineering/comments/17v3fba/chain_of_density_prompting_can_lead_to_human_level/
64. <https://smith.langchain.com/hub/majid/chain-of-density-prompt>
65. <https://advanced-stack.com/resources/how-to-summarize-using-chain-of-density-prompting.html>
66. <https://github.com/jxn/instructor/blob/main/docs/blog/posts/chain-of-density.md>
67. <https://arxiv.org/abs/2411.03538>
68. <https://arxiv.org/abs/2505.10089>
69. <https://aclanthology.org/2024.emnlp-main.1259/>
70. <https://aclanthology.org/2025.findings-emnlp.849.pdf>
71. <https://www.meilisearch.com/blog/rag-vs-long-context-llms>